Exercise: Wine classification

1. Download the dataset from https://archive.ics.uci.edu/ml/datasets/Wine. Read the description of the dataset.
2. Load the dataset. Then, based on your X and y, answer the following questions:
   - How many records are available?
   - Are there missing values?
   - How many elements does each class contain?
3. Create a DecisionTreeClassifier object with the default configuration (i.e. without passing any parameters to the constructor). Train the classifier using your X and y.
4. Now visualize it using plot_tree(), plots the tree in a matplotlib-based, interactive window. After you successfully plotted a tree, you can take a closer look at the result and draw some conclusions. In particular, what information is contained in each node?
5. Given the dataset X, you can get the predictions of the classifier (one for each entry in X) by calling the predict() of DecisionTreeClassifier. Then, use the accuracy_score() function (which you can import from sklearn.metrics) to compute the accuracy between two lists of values (y_true, the list of "correct" labels, and y_pred, the list of predictions made by the classifier). Since you already have both these lists (y for the ground truth, and the result of the predict() method for the prediction), you can already compute the accuracy of your classifier.
   - What result do you get?
   - Does this result seem particularly high/low?
   - Why do you think that is?
6. Now, we can split our dataset into a training set and a test set. We will use the training set to train a model, and to assess its performance with the test set. Sklearn offers the train_test_split() function to split any number of arrays (all having the same length on the first dimension) into two sets. You can use an 80/20 train/test split. If used correctly, you will get 4 arrays: X_train, X_test, y_train, y_test.
7. Now, train a new model using (X_train, y_train). Then, compute the accuracy with (X_test, y_test).
   - How does this value compare to the previously computed one?
   - Is this a more reasonable value? Why?
   - Compute other metrics (e.g. precision, recall, F_1 score) using the respective functions (precision_score, recall_score, f1_score).
8. So far, you have only used "default" decision trees (i.e. decision trees using the default configura- tion). The "default" decision tree might not be the best option in terms of performance to fit our dataset. In this exercise, you will perform a "grid search": you will define a set of possible config- urations and, for each configuration, build a classifier. Then, you will test the performance of each classifier and identify that configuration that produces the best model.

   On the official documentation for DecisionTreeClassifier you can find a list of all parameters you can modify. Identify some of the parameters that, based on your theoretical knowledge of decision trees, might affect the performance of the tree. For each of these parameters, define a set of possible values (the official documentation provides additional

information about the possible values that can be used). For example, we can identify these two parameters:

- max_depth, which defines the maximum depth of the decision tree, can be set to None (i.e. unbounded depth), or to values such as 2, 4, 8 (we already know from previous exercises the approximate depth the tree can reach with this dataset)
- splitter, which can be set to either best (in which case, for each split, the algorithm will try all possible splits), or random (in this case, the algorithm will try *N* random splits on various features and select the best one)

9. In the previous exercise, you searched for the best configuration among a list of possible alternatives. Since we used our test data to select the model, you may be overfitting on the test data (you may have selected the configuration that works best for the test set, but which may not be as good on new data). Typically, you do not want to use the test set for tuning the model's hyperparameters, since the test set should only be used as a final evaluation.

   For this reason, datasets are typically splitted into

   - *Training set*: used to create the model.
   - *Validation set*: used to assess how good each configuration of a classifier is.
   - *Test set*: used at the end of the hyperparameter tuning, to assess how good our final model is.

   However, it often happens that only a limited amount of data is available. In these cases, it is wasteful to only use a small fraction of the dataset for the actual training. In these cases, *cross-validation* can be used to "get rid" of the validation set. One popular method of is the *k-fold cross-validation*. In this, the training set is split into *k* partitions. *K*-1 are used for the training, the other one is used validation. This is repeated until all partitions have been used once for validation.

   Sklearn offers the KFold class for doing k-fold cross-validation. You can use this class.

   For each fold, you can use the training data to train each classifier (i.e. decision trees with different configurations) and measure the performance on the validation data. You can then aggregate the information extracted (e.g. by computing the overall accuracy from the accuracies on each fold) and select the best performing model. After you select one model, you can assess its performance on Test set.